Kalyan — Speaker Notes

Slides 9 and 10, plus narrating the live demonstration · 1 min 40 sec of slides + 2 min 20 sec of demo

Slide 9 — Database design decisions

40 seconds

Top of the slide: SQLite · 8 entities · 6 relationships · 9 business rules

Why SQL, not NoSQL

- Our data is strongly relational — a cart line is meaningless without its cart and its product.
- Foreign keys let the database itself refuse an orphaned line.
- A UNIQUE constraint on (cart_id, product_id) enforces "one line per product" with no application code.
- Totals and GST fall out of a SQL JOIN naturally.

Why SQLite specifically

- Zero configuration — the database is one file, so everyone runs the same thing after one npm command.
- Fully relational: foreign keys, CHECK and UNIQUE all supported.
- Each test file makes its own temporary database, so tests never touch the real one.

Say the trade-off before anyone asks. "SQLite handles one writer at a time. For a single-store prototype that is fine. A multi-store deployment would move to MySQL or PostgreSQL, and the schema would port directly." Naming your own limitation is what separates a Distinction from a High Distinction.

Slide 10 — Entity relationship diagram

60 seconds · worth 5 marks

The joint highest-scoring section in the rubric. Walk the diagram, do not read the boxes.

What to say

- "Eight entities. Three for the shop — products, carts, cart_items. Two for orders. Three for accounts."
- "`cart_items` is the junction table. It resolves the many-to-many between carts and products — one cart holds many lines, one product appears in many carts."
- "`order_items` does the same job for orders."

The point that impresses. "`order_items` copies product_name and unit_price at checkout. If a product's price changes next week, the order still shows what the customer actually paid — the historical record stays correct." That is a real design decision, not a textbook answer.

If asked why a box has no line

- Two relationships are left undrawn for readability: USERS to ORDERS, and USERS to PASSWORD_RESETS. Both are one-to-many.
- `revoked_tokens` stands alone — it stores a logged-out token id, not a link to a user.

Slide 19 — Live demonstration

Diya clicks · you narrate

Diya operates the browser because she built the cart interface. You say what is happening in the database.

1	“Eight products, read from SQLite. Nothing is hard-coded in the page.”
2	“The cart now belongs to this account — `carts.user_id` is set.”
3	“Two new rows in `cart_items`, linked to that cart.”
4	“A PATCH went to the API. The line total and GST came back recalculated.”
5	“The row is gone from `cart_items`. Not hidden — deleted.”
6	“Stock is three. The server refused the fourth with a 409, and the table still says three.”
7	**“One click, three tables: an orders row, order_items copying the prices, and the stock dropping. All in one transaction.”**
8	“The same order, read back through the API.”

Step 7 is your moment. Slow down. Let Diya switch the table dropdown through `orders`, then `order_items`, then `products` while you talk. One click changing three tables atomically is the strongest evidence in the whole presentation.

Your Q&A questions

Q4 How many entities are in your ERD? — **Eight.** products, carts, cart_items, orders, order_items, users, password_resets, revoked_tokens.

Q5 What is a primary key? — A column that uniquely identifies each row and cannot be null. `product_id` is the primary key of products.

Q6 What is a foreign key? — A column pointing at another table's primary key, so the database enforces that the reference is real. `cart_items.cart_id` points at `carts.cart_id`. **Add:** we switch this on explicitly with `PRAGMA foreign_keys = ON`, because SQLite does not enforce it by default.

Q7 Explain one relationship. — carts to cart_items is one-to-many: one cart holds many lines, each line belongs to exactly one cart.

Q8 How does the database support the application? — It holds every product, price, stock level and order, and enforces the rules itself: CHECK on quantity and price, foreign keys for integrity, UNIQUE COLLATE NOCASE on usernames and emails. Even if the API had a bug, the database would refuse bad data.

Q12 Why this technology stack? — Relational data, so SQL. SQLite for zero configuration and per-test databases. Node and Express because the team already knew JavaScript from the front end, so one language covers the whole stack.

Q21 and Q22 are personal. Your contribution: Week 2 data analysis, GST in the cart API, and the Sprint 4 integration — PR #46, #49, #53. Have your own answer for the hardest task, naming what you tried and what finally worked.